Java

VS

Java

Profiling Different Versions of Java
& Different methods in Java

Gurodishtzer Aaron
Hadad Eyal

Table of Contents

Overview

In this modern world there are various ways to do the exact same thing, you can use programing languages to develop different applications, algorithms, etc.

While researching on the subject we came a cross few researches which did a benchmarking test between Java and C language, and it is not hard to guess which will perform better but still, it curios to find by how much, also since Kotlin came to replace Java at Android app development we also found a comparison between these two, and this is interesting since one came to replace another, the result from the paper were – " The differences are in fact usually not very substantial, however, a common trend of Kotlin always being out-performed by Java is observed, even if that is not by a very big factor in the vast majority of benchmarks "[4]

When comparing two different programing languages, you will choose the best and most efficient way to implement the program in each language, and the result will decide which programing language will win in this contest.

But how you will know you chose the best approach to implement this ?

We are trying to figure out what is the best approach to do something since there are various ways of doing it.

Over the years since Java was released in May 1995, a lot of changes and features has been adding to it, stays the same in the aspect of high-level and object-oriented and run on the Java Virtual Machine (JVM).

There are currently 18 Java versions and, a new version is scheduled for every 6 months [2], we decided to start our testing from Java 8 since legacy project companies still using Java 8 or actually stuck with it.
There are 3 Long Term Support (LTS) Java versions, and the main focus will be on them.

- *Java 8, 11, 17.*

Each version come with a set of features some are an addition to tool set and some come to replace existing methods, we list some of the features but definitely not all and we are do not intend to, since it is out of scope for this paper.
You can read about the new features that each version is adding to the Java language, since the JDK is a superset of the JRE that have inside it the JRE some versions mostly updating the JVM and the garbage collector (GC), we will not cover these changes.

We will check the performance of *"doing the same thing just in a different way"*, on the different versions of Java. Also check the same code on different versions of Java Development Kit (JDK) and Java Runtime Environment (JRE).

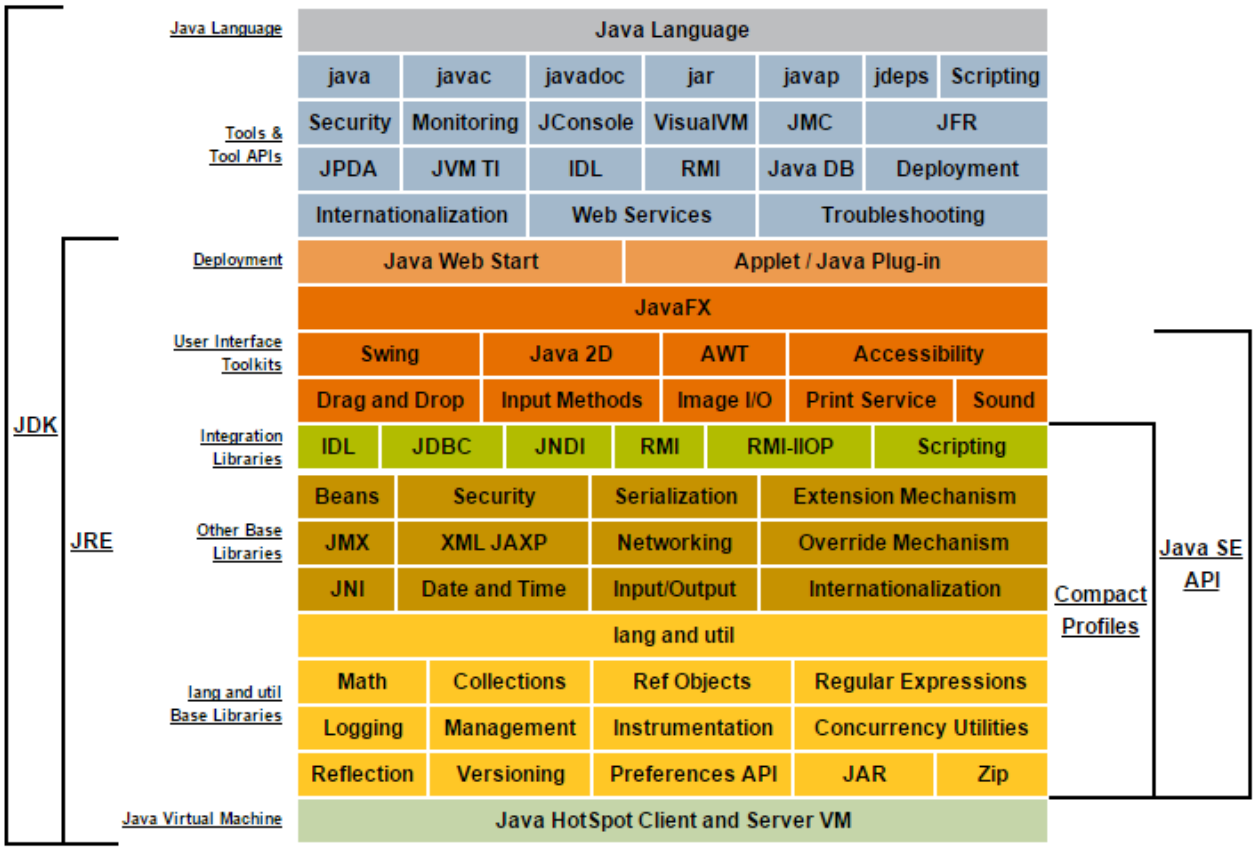
JRE:

It is a package of everything necessary to run a **complied** Java program, including the Java Virtual Machine (JVM), the Java Class Library, the *java* command, and other infrastructure.
However, it cannot be used to create new programs.

JDK:

The JDK is the full-featured SDK (Software Development Kit) for Java.
It has everything the JRE has, but also the compiler (*javac*) and tools (like *javadoc* and *jdb*).
It is capable of creating and compiling programs.

Description of Java Conceptual Diagram



[1]

Hardware details:

Most servers run on Ubuntu Linux, and not always have the latest version of the OS, we decided to run these tests also on two different machines, since legacy project companies still running on Java 8, it is making sense that there is server out there that most likely do not use the latest version, and even the one before that.
We will run on the Amazon Web Services (AWS) and use instances from the Elastic Compute Cloud (EC2), here is a quote from their website that should explain some:

> *"Amazon Elastic Compute Cloud (Amazon EC2) provides scalable computing capacity in the Amazon Web Services (AWS) Cloud.*
> *Using Amazon EC2 eliminates your need to invest in hardware up front, so you can develop and deploy applications faster.*
> *You can use Amazon EC2 to launch as many or as few virtual servers as you need, configure security and networking, and manage storage.*
> *Amazon EC2 enables you to scale up or down to handle changes in requirements or spikes in popularity, reducing your need to forecast traffic."* [5]

In this project we use 2 EC2 instances, one run Ubuntu 22.04 and the other Ubuntu 18.04 this difference is also a parameter we are comparing since there servers that still using Ubuntu 18.04 version.

We connected to these instances to run a jar file that contains the code that we would run per case that we are observing and using YourKit Java profiler in order to monitor the JVM in the remote instance.

We will run on two different hardware, the AWS EC2 hardware and our local machine Hardware – it has big gap in difference.

Local Machine:
Processor: *11th Gen Intel® Core™ i7-11800, [8-cores]*
16.0 GB RAM, (15.7 GB usable)

AWS EC2 Instance:
Processor: *t2.micro [1-core]*
1 GB RAM, (974 MB usable)

Since the EC2 instance has only 1 – core it will affect its performance, this will be perfect for our comparing since we should really feel the difference when using difference JDK since there are big upgrade involving the GC (Garbage Collector), needless to say, servers definitely have better hardware than the EC2 instance, and for that reason only we are running on our local machine.

Ubuntu Setup

For each machine before running any Java program we had to do minor changes.
install JDK for each version and also, we needed to find a way to quickly switch between versions.

After the machine is up & running do this steps.

1. *sudo apt update && sudo apt upgrade -y*

The -y flag just so you will not need to press Y when there is a confirmation of update that need to be install. But these updates are necessary in order to install any JDK version in both OS version.

Once finished updating. Installing the JDK versions. Order is irrelevant.

2. *sudo apt install openjdk-<>-jdk-headless*

In the diamond brackets need to specify the number of version you want to install we install only 8,11,17 for this research.

Since we will not use ant GUI in our performance testing, we install the headless versions.

3. need to find a way to switch quickly between versions.

    sudo update-alternatives --config java

this command doses the trick very efficiently. Just read the instructions and you are good to go.

Copy An Array

Usually when you written probably you use an array as a simple data structure, and once in a while you need to work on the data with a non-destructive approach, hence you will copy the data.

There are various ways to copy an array in Java, we chose the most common ways.

1. for loop – the intuitive way
2. Using the '*java.lang.Object.clone()'*
3. Using *Arrays.copyOf()* - '*java.utils'*
4. Using *'java.lang.System.arraycopy()'*
5. for each loop – mostly used on Data Structures

~ Note

We ran each method with a different class and one main thread that calling each method at a time, to compare the usage of the CPU respectively.
The order is as mentioned above, it matters ! look at Heap Image

---

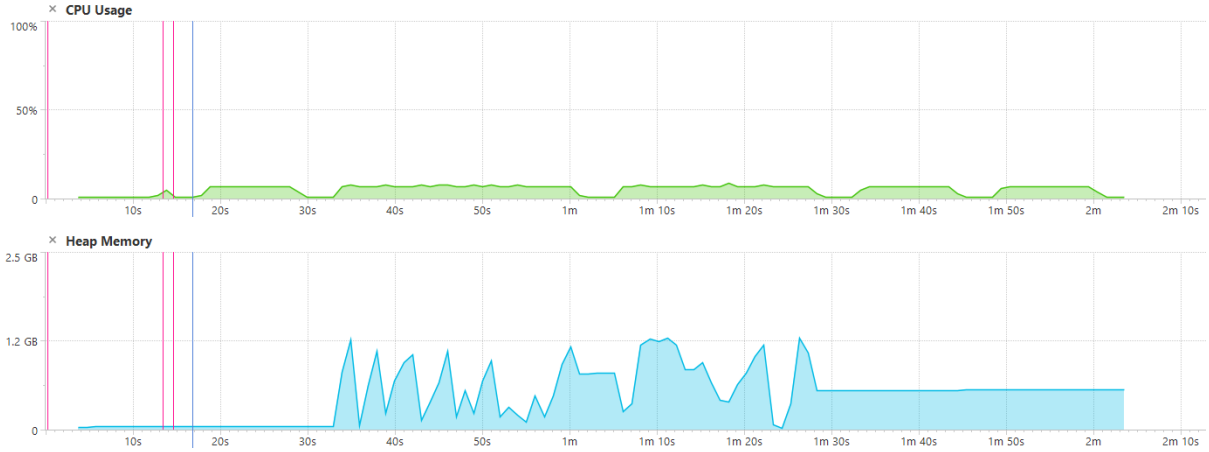
Starting with *Windows 10* example just to get a perspective of the *copy-Array* performance Than we will run on our remote machine to see the difference when we have only one core and 1-GB RAM.

Note.
This note is for all performance tests we will run on the remote machine – since there is only one core, when attaching the profiler to the Java program we will see a lot of the time differences since there will be a lot of *CONTEX-SWITCHING* in order for the CPU to give each Thread the required time. We will take that parameter in count so when reproducing our result please take that in mind.

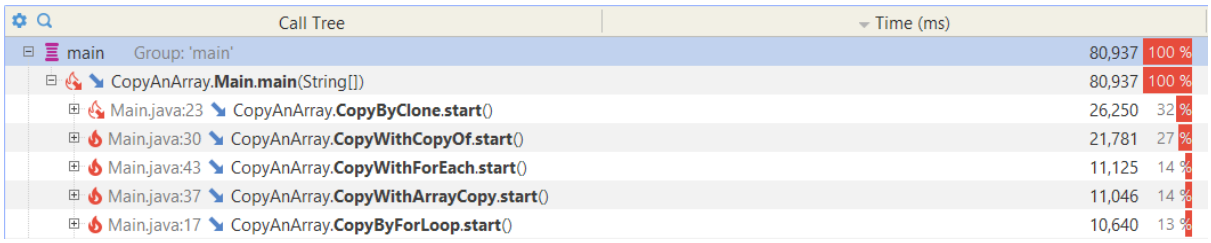
Steps to reproduce:

1. create 16 arrays two for each primitive type SIZE = 100,000:
   byte, short, int, long, float, double, char, boolean
2. fill 8 arrays, each one with a supported value, randomness is redundant it can be the same value for each cell of the array.
3. Create a for-loop that will run 100,000 inside a separate class that will have access to the 16 arrays mentioned in step (1)
4. In each iteration Copy each full array to an empty one – after the first iteration both arrays will be full but that doesn't matter.
5. You should have by now 5 classes each one with a for-loop inside that for loop 8 arrays that being copied
6. We run everything from 1 Main Thread, 5 seconds sleep between each class.

Windows 10, Local Machine – JAVA 8

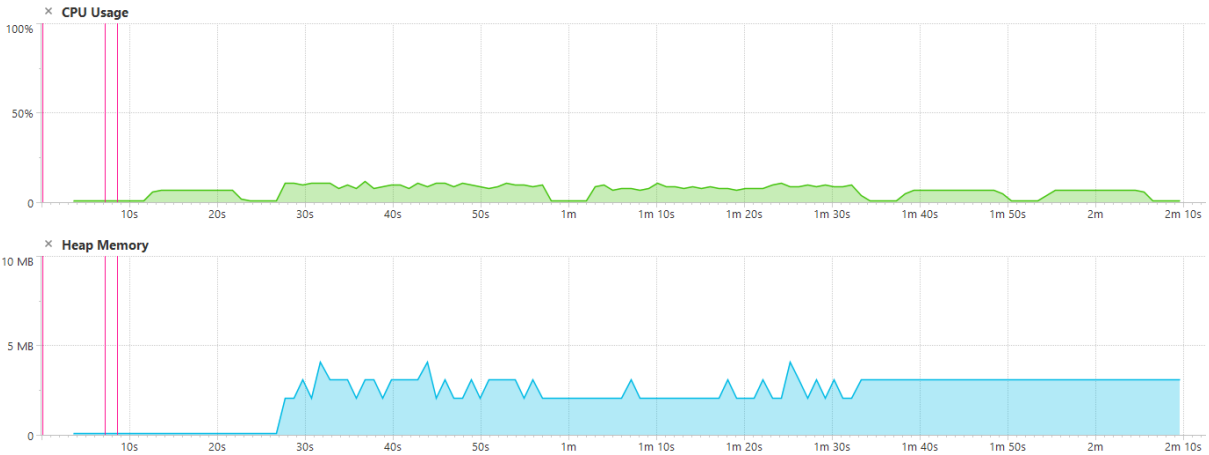


We can see the Heap-memory usage when reaching to 2,3 methods, no usage at all for the other methods (1,4,5), It explains the CPU usage results (of the Main thread).
~when changing the order of method 2 and 3 the Heap usage is similar.



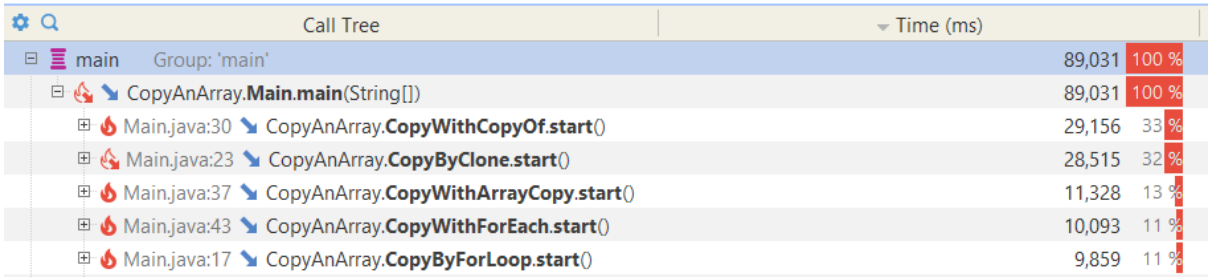
The 1,4,5 methods kept an average time 10.5 seconds, the 4 method use slightly more the CPU by 1%. The others kept the gap as it is.

Windows 10, Local Machine – JAVA 11



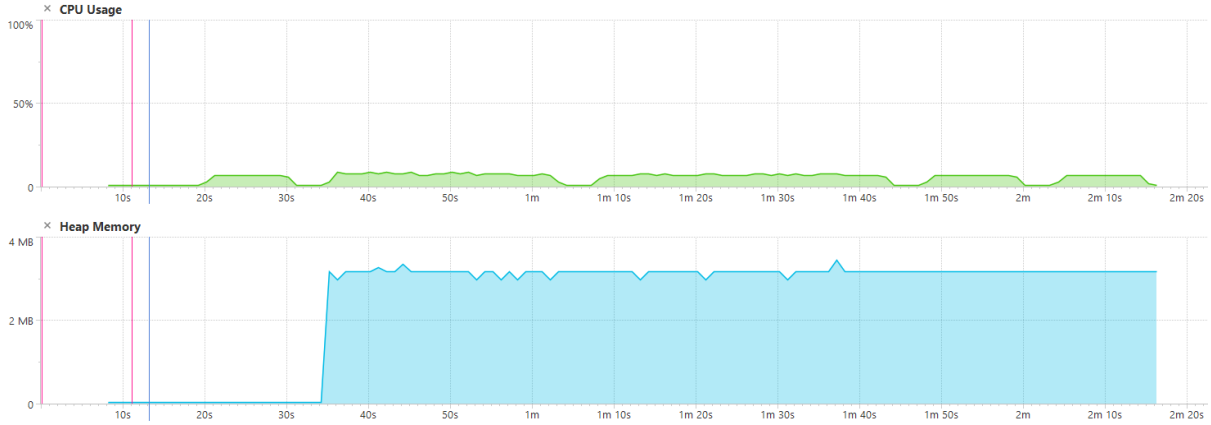
Same as Java8 when reaching 2,3 the Heap-memory is used, but and there is a **BIG** but, we see a drastic change in the memory usage, the max usage in Java8 is around 1.2GB and in

Java11 not even reaching 5MB, the explanation for this is probably since in Java 9 & 10 there are updates to the GC and more updates that related to memory allocation.

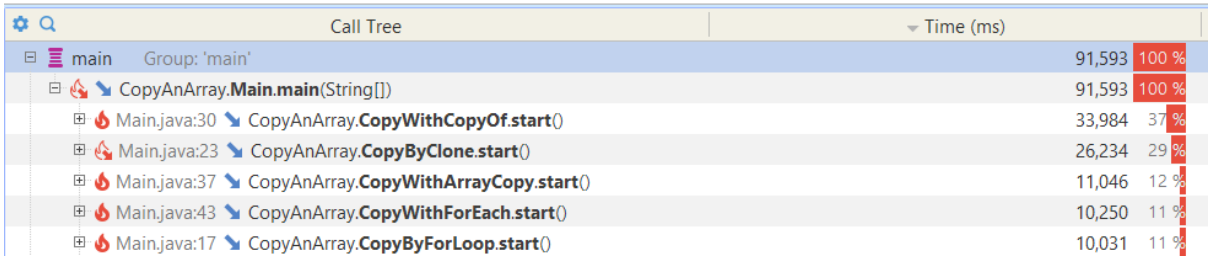


The order did not change much, methods 4,5 change places an average of 10.5 seconds but the CPU usage was less for the 5 method in most cases or equal to 4 method.

Windows 10, Local Machine – JAVA 17

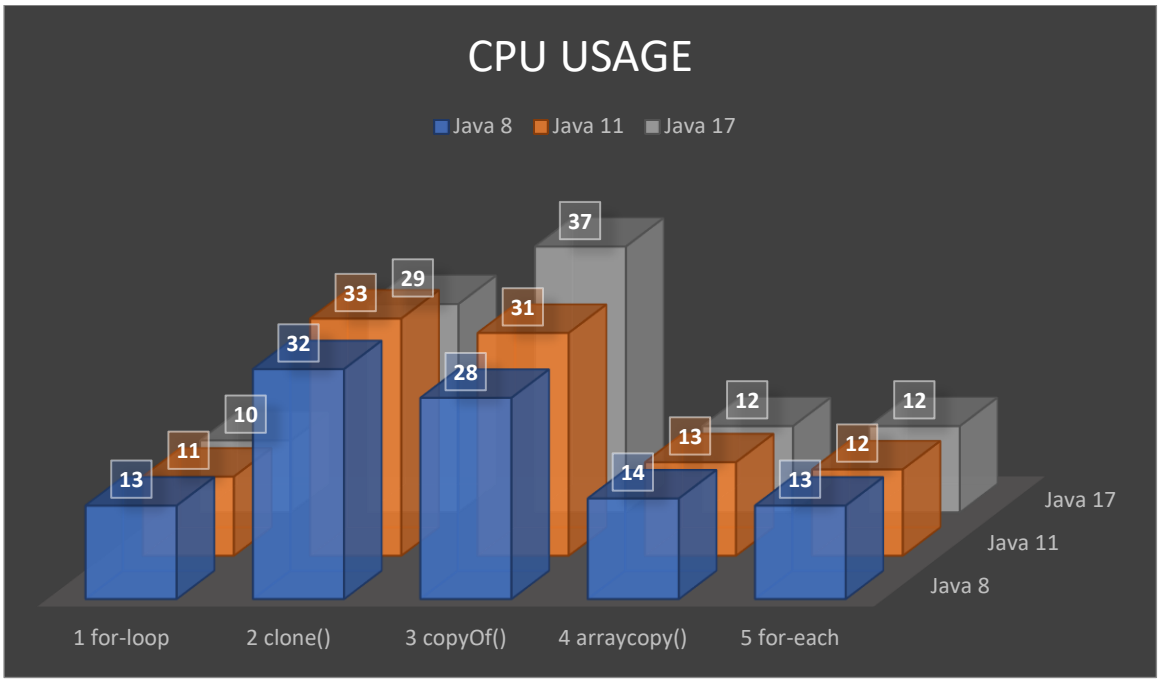


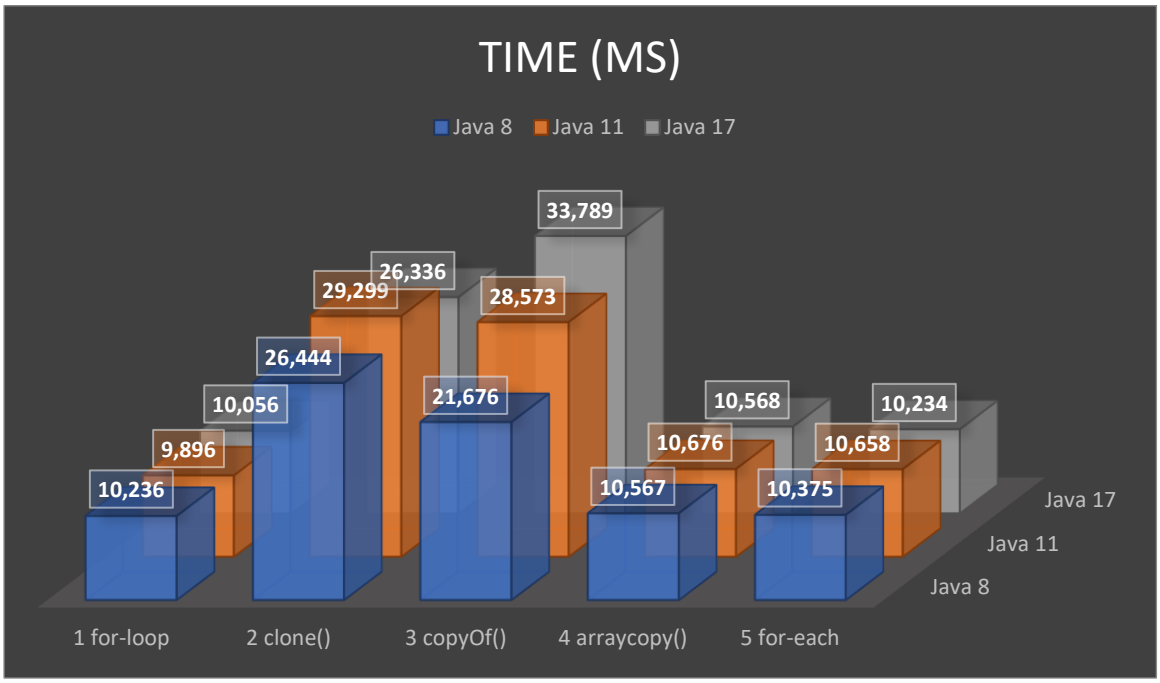
Another improvements in memory !



We see another change in events when running on Java17 this time the 3 method consumed much more than 2 method, in Java 8 & 11 the 2 method was in the top.

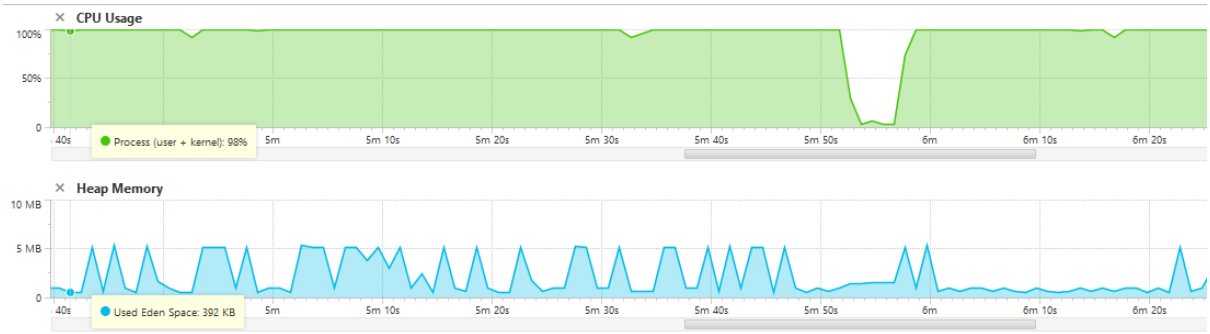
Conclusions:





Ubuntu18.04 VS Ubuntu22.04

Java 8



For both OS version the CPU usage and Heap-Memory were similar – it cannot make a real difference where we have only 1-core and 1-GB RAM.

We will check if there are improvements when running Java 11 and 17, and we will compare the

Java Streams – Powerful but which way ?

Well, whoever knows Java language should know Java Streams, you can really do a lot with it and with a few lines also, but we really could not ignore the fact that you can do the same thing in a different way – and of course get the same result.

Since our main subject is the Java language itself and not just the Stream API, we encourage you to find more ways of using Streams and get the same result – you check by yourself which is best.

We focus on 3 different ways to sum a list of object (Employee), one of the methods we divide it into 2 different ways, hence we got 4 terminal operations that conclude the same result.

1. Using Collectors

```
long integer = employees.stream() Stream<Employee>
        .map(employee -> (int) employee.age) Stream<Integer>
        .collect(Collectors.summingInt(Integer::intValue));
```

2. Reduce using lambda

```
long integer = employees.stream() Stream<Employee>
        .map(employee -> (int) employee.age) Stream<Integer>
        .reduce( identity: 0, (a, b) -> a + b);
```

3. Reduce using method reference

```
long integer = employees.stream() Stream<Employee>
        .map(employee -> (int) employee.age) Stream<Integer>
        .reduce( identity: 0, Integer::sum);
```

4. Mapping into an IntStream then summing it.

```
long integer = employees.stream() Stream<Employee>
        .mapToInt(employee -> (int) employee.age) IntStream
        .sum();
```

Record vs. Class

Record is a special class that we got to know in Java 14, it used to replace a class that act only as a plain data carrier, when you need to keep records in your data base for example, you can use class that way.

Since Java 14, Record is the new way of doing it, I suppose once you get used it, it is more convenient and your code will be much more readable and will avoid boilerplate code, but is it efficient ?

(Record was Standardized since Java 16).[1]

[1] For more information – Features in each version

What should I do Switch-Case or If and If else ?

Every programmer when dealing with multiple inputs that each one need to be handled differently, had that thought in mind – is too much if-else, should I do switch-case ?

Well switch case is usually much more readable especially when you're dealing with cases that need to handle the input the same way, with if-else it will be more complicated to achieve it, but in general what is most efficient way ?

Using CONSTANTS – Enum or Static final ?

When developing a big project, and you want to have a clean and readable code you need to have a lot of *CONSTANTS.*

The old style is to use Strings as a static members in one class of course those members need to have some relation considering they are in the same class, but sometimes you have only one String that represent something and that is it, you probably will not open a dedicated class just for one String, since Java 5 Enumerations are available to us in Java and dedicated class can be created to all those Enum`s.

But the special question needs to be asked, which is best ?

Should I use a Stream or a For loop or ForEach ?

Every programmer had to iterate over a data structure at least once in day-day developing did you ever ask yourself how to iterate over the data structure is preferable ? we ask ourselves for you.

Well having an array, you can either loop over it with 2 different ways in Java or you can use the Stream API that Java has to offer since Java 8, sometimes you just iterate over it with a simple task like to sum all its elements, and sometimes it is a bit more complex and you need to filter some of the element that does not qualify your terms.

Different way to Read \ Write Files

Well read and write files is very common wherever you look, if you want to encrypt some files, you eventually will need also to read files first and after that to write them with the new encryption but this just one example.

There are more than a few ways to do so, we tried to focus on our main Google results, and really hoped for to find a difference in the methods

Features in each version

We will list the featured that related to our paper, the number of features for each version is different and, some are more related, and some are not.

- Java 5 - Enumerations, ForEach loop which is called enhanced ForEach loop. [3]
- Java 7 - String in Switch. [3]
- Java 8 - The famous Stream API, Lambda expressions. [3]
- Java 9 - Improvement in the Stream API. [3]
- Java 12 - Switch Expressions (Preview). [3]
- Java 13 - Switch Expressions enhancement. [3]
- Java 14 - Switch Exp` (Standard), Records (Preview), *instanceOf* pattern match (Preview). [3]
- Java 16 - Records (Standard), *instanceOf* pattern matching (Standard). [3]
- Java 17 - Pattern Matching for switch (Preview). [3]
- Java 18 - Pattern Matching for switch (Preview). [3]

As mentioned above we did not list all the features for each version, and we ignore some. The are many features which related to web development and the JVM engine, these features are out of scope for this paper.

References

[1] https://stackoverflow.com/questions/1906445/what-is-the-difference-between-jdk-and-jre

[2] https://www.marcobehler.com/guides/a-guide-to-java-versions-and-features

[3] https://howtodoinjava.com/java-version-wise-features-history/

[4] http://www.diva-portal.org/smash/record.jsf?pid=diva2%3A1443070&dswid=-2576

[5] https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/concepts.html